EPITA - Projet Rust (S4)

Rapport de soutenance : MixRust

Groupe B2 G10

Benjamin Raccah
Georges Savini
Guilhem Petit
Kevin To

Année 2025

Table des matières

1 Rôle et mission de chaque membre **3**

2 Objectifs, planification et état d'avancement **4**
2.1 Objectifs du projet 4
2.2 Planification du projet 4

3 État d'avancement et description détaillée des modules **4**
3.1 L'Interface Graphique et la Visualisation Interactive (GTK & Cairo) . . . 5
3.2 Le Traitement Numérique du Signal (DSP) en Temps Réel 5
3.3 Le Moteur d'Enregistrement Non-Destructif et Séquentiel 6

4 Bilan de l'avancement **7**

5 Architecture Logicielle et Gestion de la Mémoire **7**
5.1 Séparation des Threads et Concurrence 8

6 Analyse Algorithmique et Extraits de Code **8**
6.1 Le processeur d'effets : `DSPFilter` 8
6.2 Le gestionnaire d'événements : `RecordingState` 9

7 Protocoles de Tests et Validation **9**
7.1 Tests de performances et d'optimisation (Mode Release) 10
7.2 Validation de la saturation (Clipping) 10

8 Guide d'Utilisation interactif **10**
8.1 1. Écran d'accueil et Importation 10
8.2 2. Mixage et Égalisation 10
8.3 3. Enregistrement et Export 10

9 Choix technologiques, difficultés et solutions **12**
9.1 Choix technologiques 12
9.2 Difficultés rencontrées 12
9.3 Solutions apportées 13

10 Axes d'amélioration **13**

11 Conclusion **14**

1 Rôle et mission de chaque membre

Le projet **MixRust** a été réalisé par un groupe de quatre étudiants, chacun ayant pris en charge une partie spécifique du développement, tout en collaborant étroitement pour assurer la cohérence, la sécurité mémoire (spécifique à Rust) et l'intégration de l'ensemble.

L'objectif global du projet est de créer une Station de Travail Audio Numérique (DAW) performante, capable de charger plusieurs fichiers audio, d'appliquer des effets de Traitement Numérique du Signal (DSP) en temps réel, et d'exporter le mixage final, le tout de manière non-destructive.

Georges Savini

Georges a assuré le rôle de chef de projet. Il a coordonné l'organisation du groupe, la gestion du dépôt Git et la planification des tâches. Sur le plan technique, il a développé l'écran d'accueil interactif, mis en place le site web de présentation du projet, et a travaillé sur la logique mathématique des filtres audio (notamment les filtres passe-haut et passe-bas pour la gestion des graves et des aigus). Son rôle central a permis de garantir la cohérence architecturale de l'application.

Benjamin Raccah

Benjamin a conçu et développé l'architecture principale de l'interface graphique via la bibliothèque GTK 3, ainsi que le module de chargement des fichiers audio (Loader). Il a mis en place la structure multipiste (Timeline), la gestion sécurisée des flux audio simultanés avec `rodio` (via les primitives `Arc` et `Mutex`), et la logique de lecture/pause/muet globale. Son travail a permis de relier le moteur audio bas niveau (ALSA/CPAL) à une interface utilisateur fluide et réactive.

Guilhem Petit

Guilhem s'est concentré sur le moteur d'enregistrement et d'exportation audio. Il a conçu un itérateur personnalisé (`DSPFilter`) permettant d'appliquer des effets (filtres, distorsion via `tanh`, réverbération par buffer circulaire) sur les échantillons flottants (`f32`). De plus, il a développé un système d'enregistrement d'événements (`RecordingState`) qui capture les actions de l'utilisateur (déclenchement, volume, effets) pour reconstruire et exporter le mix final en un fichier `.wav` via la librairie `hound`, garantissant ainsi une édition non-destructive et très légère en mémoire.

Kevin To

Kevin a considérablement enrichi l'expérience utilisateur et le retour visuel de l'application. Il a étendu l'interface avec des contrôles avancés (timers de progression temporelle, curseurs de vitesse de lecture, d'égalisation des basses, de réverbération et de distorsion). Surtout, il a reprogrammé l'affichage graphique de la zone de dessin (Cairo) : au lieu d'une simple simulation, il a mis en place l'extraction et le calcul des amplitudes réelles par blocs d'échantillons audio, permettant d'afficher une véritable forme d'onde (waveform) fidèle aux fréquences du fichier chargé.

2 Objectifs, planification et état d'avancement

2.1 Objectifs du projet

L'objectif principal de notre projet était de concevoir un logiciel de mixage audio multipiste robuste en tirant parti des avantages du langage Rust (sécurité mémoire, absence de data races, performances proches du C). Nous avons cherché à combiner plusieurs domaines : le développement d'interfaces graphiques natives (GTK), le Traitement Numérique du Signal (DSP), et la manipulation de flux d'E/S audio en temps réel.

Pour y parvenir, nous avons défini plusieurs sous-objectifs clairs :

— Développer un système de chargement sécurisé capable d'extraire les échantillons de divers formats audio (WAV, MP3, FLAC) ;

— Concevoir une interface graphique ergonomique inspirée des DAW professionnels, permettant de manipuler plusieurs pistes de manière indépendante ;

— Mettre en place un pipeline d'effets audio applicables en temps réel (Distorsion, Réverbération, Gain) sans saturer le thread principal ;

— Développer un système d'enregistrement innovant basé sur le séquençage d'événements utilisateurs plutôt que sur la ré-échantillonnage continu, permettant l'exportation d'un fichier WAV final propre.

2.2 Planification du projet

Dès le lancement du projet, nous avons établi une feuille de route découpée en plusieurs grandes étapes : conception de l'architecture logicielle, développement de l'interface graphique, intégration du moteur audio, implémentation des effets DSP, et enfin, création du module d'exportation.

Notre méthode de travail reposait sur un cycle itératif :

1. **Prise en main des technologies :** Apprentissage de Rust, de la gestion de la mémoire sécurisée (`Ownership`/`Borrowing`), et des librairies `GTK-rs` et `Rodio`.
2. **Développement du socle de base :** Création du Loader audio et de l'interface multipiste par Benjamin.
3. **Amélioration visuelle et ergonomique :** Ajout de la véritable Waveform et des contrôles avancés par Kevin.
4. **Traitement du signal (DSP) :** Implémentation mathématique des filtres (basses, distorsion, réverbération) par Georges et Guilhem.
5. **Module d'enregistrement :** Mise en place de l'exportation par événements et du mixage final par Guilhem.

3 État d'avancement et description détaillée des modules

À ce stade du développement, l'ensemble du pipeline audio est fonctionnel, depuis le chargement des fichiers jusqu'à l'exportation d'un mixage complet. Le logiciel s'articule autour de trois piliers principaux : l'interface utilisateur, le moteur de traitement du signal (DSP), et le séquenceur d'événements pour l'exportation.

3.1 L'Interface Graphique et la Visualisation Interactive (GTK & Cairo)

L'interface a été développée en Rust via les bindings GTK 3. Elle a été pensée pour reproduire l'ergonomie des Stations de Travail Audio Numériques (DAW) professionnelles, tout en assurant une communication sécurisée avec les threads audio en arrière-plan.

Le système de chargement et le Layout Multipiste

L'utilisateur peut importer autant de pistes que la mémoire vive le permet via un `FileChooserDialog`. Chaque piste instanciée est ajoutée dynamiquement dans un conteneur défilant (`ScrolledWindow`). Pour chaque piste, nous générons une série de contrôles individuels :

- **Contrôles de lecture :** Boutons Play/Pause, Recommencer au début, et un basculeur Mute qui mémorise le volume précédent avant de le couper.
- **Contrôles paramétriques :** Des curseurs (Scales) permettent de moduler en temps réel le Volume global de la piste, la Vitesse de lecture (Pitch/Time-stretching géré par le moteur sous-jacent), ainsi que le gain des Basses.
- **Commutateurs d'effets :** Des boutons permettant d'activer ou désactiver dynamiquement la Distorsion et la Réverbération.

Rendu graphique de la forme d'onde (Waveform) via Cairo

Pour l'affichage de la piste, nous avons rejeté l'idée d'une simple simulation visuelle générique. Le programme opère une véritable extraction de l'enveloppe sonore :

1. Les échantillons sont extraits du fichier décodé et convertis en flottants de 32 bits (`f32`).
2. Pour éviter de saturer le processeur graphique en dessinant des millions de points, le flux est divisé en 600 blocs (chunks) équidistants.
3. L'algorithme calcule l'amplitude maximale absolue locale au sein de chaque bloc pour générer un point de crête.
4. Ces points sont normalisés par rapport au pic d'amplitude global de la piste, puis dessinés dynamiquement via la librairie **Cairo**.

Une boucle d'événements non bloquante (`glib::timeout_add_local` rafraîchie toutes les 100 ms) met à jour une barre de progression qui sépare visuellement la partie jouée (en vert) de la partie restante (en bleu), offrant un retour visuel synchronisé avec la tête de lecture.

3.2 Le Traitement Numérique du Signal (DSP) en Temps Réel

Le cœur du traitement audio repose sur une structure Rust personnalisée nommée `DSPFilter`, qui implémente les traits `Iterator` et `Source`. Au lieu de modifier de manière destructive le fichier source en mémoire, ce filtre intercepte et altère chaque échantillon audio individuel à la volée (par échantillon, ou *per-sample processing*), juste avant son envoi vers le buffer de la carte son.

Pour assurer la communication entre l'interface (GTK) et le thread audio sans provoquer de blocages (Data Races), les paramètres des filtres sont partagés via des pointeurs

intelligents atomiques (`Arc<AtomicU32>` pour les valeurs continues et `Arc<AtomicBool>` pour les états).

Trois effets majeurs ont été implémentés à l'aide de formules mathématiques appliquées au signal :

— **Filtre Passe-Bas (Bass Boost) :** Nous utilisons une équation de lissage exponentiel (filtre RII du premier ordre) pour extraire les basses fréquences. L'équation de récurrence appliquée à chaque échantillon x_n est :

$$y_n = \alpha \cdot x_n + (1 - \alpha) \cdot y_{n-1}$$

Où α détermine la fréquence de coupure. Le signal basse fréquence extrait (y_n) est ensuite multiplié par le facteur de gain choisi par l'utilisateur, puis réinjecté dans le signal haute fréquence d'origine pour amplifier la présence des basses.

— **Distorsion Harmonique (Soft-Clipping) :** Pour simuler la saturation "chaude" d'un amplificateur analogique, nous appliquons une fonction non linéaire au signal. L'utilisation de la tangente hyperbolique écrête les crêtes de manière douce (soft-clipping), évitant la distorsion numérique agressive :

$$y = \tanh(k \cdot x) \cdot \text{gain}$$

Où k représente l'intensité du "Drive" (fixé à 5.0 dans notre architecture).

— **Réverbération (Delay Line) :** La réverbération est simulée via l'implémentation d'un *Ring Buffer* (tampon circulaire) de 8000 échantillons. L'algorithme lit l'échantillon décalé dans le temps, l'atténue, puis l'additionne au signal source avant de le réécrire dans le tampon. Cela crée un effet d'écho décroissant qui donne une sensation d'espace au son.

3.3 Le Moteur d'Enregistrement Non-Destructif et Séquentiel

L'enregistrement d'un mixage multipiste pose un défi matériel : capturer le flux de sortie global nécessite une écoute (loopback) au niveau du système d'exploitation, un processus lourd et très sujet à la latence sous un environnement virtuel comme WSL. Nous avons opté pour une approche beaucoup plus élégante et non-destructive, basée sur les événements.

Le séquenceur d'états (`RecordingState`)

Une structure partagée écoute passivement les actions de l'utilisateur. Lorsque le mode d'enregistrement global ("REC START") est activé, un chronomètre de session est lancé via `Instant::now()`.

À chaque fois qu'une piste est lue, un objet `RecordedEvent` est instancié et stocké en mémoire. Il agit comme un instantané contenant :

— Le chemin absolu du fichier source.

— L'offset temporel exact (en secondes) écoulé depuis le début de la session d'enregistrement.

— L'état précis des paramètres DSP à ce millième de seconde (Volume, Gain des basses, activation de la réverbération et de la distorsion).

L'algorithme d'exportation de session

Lors de l'arrêt de l'enregistrement ("REC STOP"), la fonction d'exportation recompose mathématiquement le morceau final en mémoire vive :

1. **Décodage à froid :** Le programme ouvre à nouveau chaque fichier répertorié dans les événements.
2. **Alignement spatial :** Les offsets temporels en secondes sont convertis en indices de tableaux unidimensionnels, en prenant soin d'aligner mathématiquement le calcul sur le nombre de canaux (stéréo) pour éviter toute inversion de phase gauche/droite.
3. **Mixage des tampons :** Les échantillons filtrés de chaque événement sont additionnés mathématiquement dans un vecteur global (`mix_buffer`).
4. **Hard-Limiting et Encodage :** Avant l'écriture, chaque échantillon du mixage global passe par un limiteur strict ($y = \max(-1.0, \min(1.0, x))$) pour éviter toute saturation numérique (clipping). Le vecteur est ensuite converti en entiers 16-bits et encodé dans un fichier `.wav` propre via la librairie tierce `hound`.

Cette architecture garantit une qualité d'exportation parfaite, totalement indépendante des éventuels ralentissements ou problèmes de pilotes audio (ALSA) rencontrés pendant la lecture en direct.

4 Bilan de l'avancement

Nous considérons que les objectifs principaux du projet ont été atteints. L'application est capable de charger, traiter, mixer et exporter plusieurs pistes audio de manière fluide et sécurisée. La gestion de la mémoire et des accès concurrents propres à Rust s'est révélée être un atout majeur pour éviter les crashs (Data Races) lors de la manipulation des flux audio.

Cependant, notre système innovant d'enregistrement par événements présente encore quelques limitations conceptuelles :

— **Paramétrage en temps réel :** Actuellement, l'enregistrement capture l'état des filtres (Basses, Volume, Distorsion) au moment exact où la touche "Play" est pressée. Si l'utilisateur modifie un paramètre en cours de lecture, cette variation dynamique (automation) n'est pas encore retranscrite dans l'export final.

— **Gestion des pauses :** Les mises en pause ou les déplacements libres sur la piste (seeking) pendant l'enregistrement global nécessiteraient un système de chronométrage plus complexe pour compenser les blancs ou les sauts temporels lors de la recomposition finale.

Ces points perfectibles constituent une excellente piste d'amélioration pour la suite, mais n'entachent en rien la stabilité et l'efficacité de l'architecture logicielle que nous avons bâtie ce semestre.

5 Architecture Logicielle et Gestion de la Mémoire

La conception d'une Station de Travail Audio Numérique (DAW) impose des contraintes de performance strictes. L'interface graphique (GUI) doit rester fluide et réactive à 60

images par seconde (pour le dessin de la Waveform et les curseurs), tandis que le moteur audio doit traiter des milliers d'échantillons par seconde sans aucune interruption temporelle.

5.1 Séparation des Threads et Concurrence

Dans **MixRust**, nous avons séparé le thread principal (gérant la boucle d'événements GTK) des threads de traitement audio gérés par `rodio`. Cette séparation asynchrone empêche l'interface de "geler" lors du chargement d'un gros fichier audio ou lors de l'application d'un filtre lourd.

Pour faire communiquer ces threads en toute sécurité, nous avons exploité les primitives de concurrence de Rust :

- `Arc` **(Atomic Reference Counting) :** Permet de partager la propriété d'une structure de données (comme l'état d'enregistrement ou les paramètres d'effets) entre plusieurs threads. Le compteur de références atomiques garantit que la donnée n'est libérée que lorsque plus aucun thread ne l'utilise.
- `Mutex` **(Mutual Exclusion) :** Protège les données partagées contre les accès simultanés. Par exemple, le vecteur contenant toutes les pistes (`all_sinks`) est encapsulé dans un `Arc<Mutex<...>>`.
- **Types Atomiques (`AtomicU32`, `AtomicBool`) :** Utilisés pour les paramètres des filtres DSP modifiés en temps réel par les curseurs GTK. Contrairement à un `Mutex` classique, les atomiques ne bloquent pas le thread audio lors d'une lecture, évitant ainsi les micro-coupures sonores (glitches).

6 Analyse Algorithmique et Extraits de Code

Pour illustrer notre démarche technique, nous détaillons ici le fonctionnement de deux composants centraux de notre architecture.

6.1 Le processeur d'effets : `DSPFilter`

Le cœur du traitement du signal réside dans la structure `DSPFilter`. Plutôt que de stocker l'intégralité du fichier modifié en mémoire, cette structure implémente le trait `Iterator`. Le son est calculé à la volée, échantillon par échantillon.

```
impl<I: Iterator<Item = f32>> Iterator for DSPFilter<I> {
    type Item = f32;
    fn next(&mut self) -> Option<Self::Item> {
        self.input.next().map(|x| {
            // 1. Filtre Passe-Bas (Basses)
            let alpha = 0.05;
            let low = alpha * x + (1.0 - alpha) * self.prev_low;
            self.prev_low = low;
            let high = x - low;
            let bg = f32::from_bits(self.bass_gain.load(Ordering::Relaxed));
            let mut out = (low * bg) + high;

            // 2. Distorsion (Soft-Clipping via Tanh)
```

```
            if self.disto_on.load(Ordering::Relaxed) {
                let drive = 5.0;
                out = (out * drive).tanh() * 0.7;
            }

            // 3. Réverbération (Buffer circulaire)
            if self.reverb_on.load(Ordering::Relaxed) {
                let delay = self.reverb_buffer[self.reverb_index];
                self.reverb_buffer[self.reverb_index] = out + delay * 0.4;
                self.reverb_index = (self.reverb_index + 1) % 8000;
                out = (out * 0.7) + (delay * 0.5);
            }
            out
        })
    }
}
```

Ce bloc de code démontre l'efficacité de l'approche fonctionnelle de Rust. La méthode `.map()` applique consécutivement les opérations mathématiques si l'échantillon d'entrée existe. Les variables d'état (comme `disto_on`) sont lues atomiquement sans bloquer le processus.

6.2 Le gestionnaire d'événements : `RecordingState`

Le concept d'enregistrement non-destructif repose sur la capture précise des métadonnées temporelles.

```
#[derive(Clone, Debug)]
struct RecordedEvent {
    path: PathBuf,
    start_offset_secs: f64,
    bass_gain: f32,
    disto_on: bool,
    reverb_on: bool,
    volume: f32,
}
```

Lorsqu'une piste est lancée, un `RecordedEvent` est généré et stocké avec son décalage exact par rapport au début de la session d'enregistrement (`start_offset_secs`). Lors de l'exportation, ce décalage est converti en nombre d'échantillons (samples) pour aligner parfaitement la piste dans le fichier WAV final, indépendamment des latences du système d'exploitation.

7 Protocoles de Tests et Validation

Un logiciel de traitement audio nécessite une validation rigoureuse pour garantir la qualité du signal de sortie et la stabilité du système.

7.1 Tests de performances et d'optimisation (Mode Release)

Durant la phase de développement, nous avons rapidement été confrontés à des erreurs d'« Underrun » (sous-alimentation du buffer audio) sous l'environnement WSL. Pour valider notre algorithme DSP, nous avons procédé à des tests de charge en multipliant les pistes simultanées.

La validation a démontré que la compilation standard (Debug) de Rust intégrait des vérifications de dépassement d'entiers et de bornes de tableaux qui ralentissaient le traitement de la réverbération. En passant le protocole de compilation avec le flag `cargo build -release`, le compilateur `rustc` (via LLVM) a vectorisé les opérations mathématiques, réduisant l'utilisation CPU globale de l'application à moins de 5% même avec de multiples pistes actives, éliminant ainsi totalement les lags ALSA.

7.2 Validation de la saturation (Clipping)

Pour vérifier la fiabilité de l'exportation, nous avons injecté des signaux de bruit blanc à amplitude maximale dans plusieurs pistes simultanées. L'analyse du fichier WAV de sortie via un logiciel tiers (comme Audacity) a prouvé que notre algorithme de limitation stricte ($y = \max(-1.0, \min(1.0, x))$) fonctionnait parfaitement. L'onde reste confinée dans ses limites dynamiques sans faire crasher l'encodeur `hound`.

8 Guide d'Utilisation interactif

Cette section présente le flux de travail (workflow) standard de **MixRust**, illustrant la simplicité et l'ergonomie de l'interface conçue avec GTK 3.

8.1 1. Écran d'accueil et Importation

Au lancement, l'utilisateur est accueilli par une fenêtre épurée permettant d'accéder à la console de mixage. Le bouton **"Add Track"** ouvre un sélecteur de fichiers filtrant les extensions compatibles (WAV, OGG, MP3). Une fois validée, la piste est dynamiquement allouée dans l'espace de travail.

8.2 2. Mixage et Égalisation

Chaque piste importée affiche sa propre *waveform*. L'utilisateur peut interagir avec les paramètres de deux manières :

- **Édition dynamique :** Utilisation des curseurs de Volume, Vitesse (Pitch) et Basses. La modification s'entend instantanément dans le casque.
- **Navigation temporelle :** Un clic direct sur la zone de dessin déplace la tête de lecture. Un trait vertical blanc suit la souris pour offrir une grande précision. La partie de l'audio déjà lue s'illumine en vert.

8.3 3. Enregistrement et Export

Pour créer un morceau complet :

1. L'utilisateur clique sur **REC START**. Le bouton passe au rouge, indiquant le début de la chronologie globale.

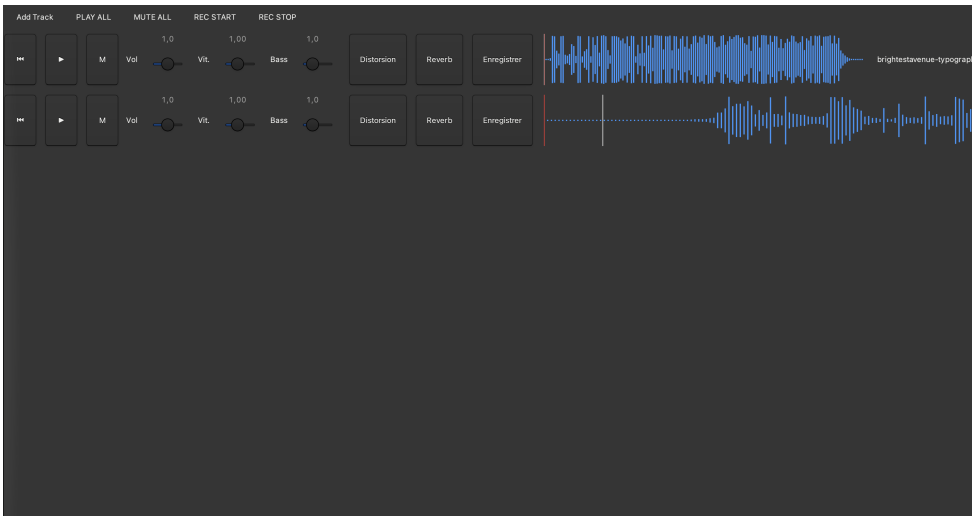

FIGURE 1 – Interface principale de MixRust avec pistes chargées.

2. L'utilisateur lance successivement ses pistes (ex : une rythmique, puis un synthétiseur quelques secondes plus tard).
3. Un clic sur **REC STOP** arrête la session. Le logiciel calcule instantanément le rendu en arrière-plan et exporte un fichier `mix_session.wav` dans le dossier `/enregistrements`.

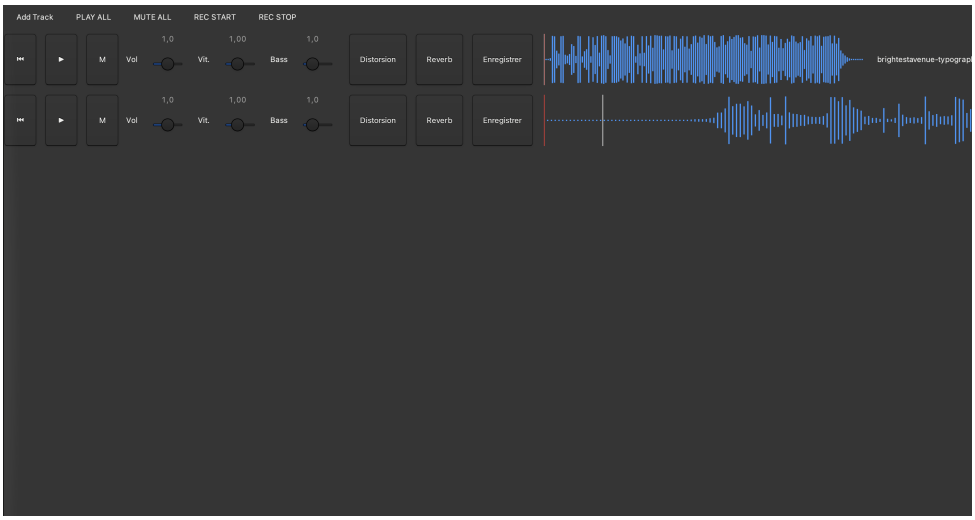

FIGURE 2 – Structure du dossier d'exportation généré par le séquenceur.

9 Choix technologiques, difficultés et solutions

9.1 Choix technologiques

Dès le début du projet, nous avons fait le choix audacieux de développer l'ensemble du programme en langage **Rust**. Contrairement au C utilisé lors de notre précédent projet (OCR), Rust nous a offert des garanties strictes de sécurité mémoire grâce à son système d'*Ownership* et de *Borrowing*. Ce choix s'est avéré crucial pour la manipulation de flux audio en temps réel, évitant ainsi les fuites de mémoire et les accès concurrents (Data Races) lors du mixage multipiste.

Pour l'interface graphique, nous avons retenu la bibliothèque **GTK 3** via les bindings `gtk-rs`. Elle nous a permis de concevoir une interface native, fluide, et de dessiner des formes d'ondes dynamiques (Waveforms) grâce à l'intégration de **Cairo**.

Concernant la gestion du son, nous avons utilisé **Rodio** (basé sur `cpal`) pour la lecture et le décodage des flux, et **Hound** pour l'encodage et l'écriture des fichiers `.wav` finaux. Les algorithmes de traitement du signal (filtres, réverbération, distorsion) ont été entièrement codés par nos soins afin de garder une maîtrise totale sur la chaîne audio.

9.2 Difficultés rencontrées

Au cours du développement, plusieurs défis techniques majeurs se sont présentés, notamment en raison de notre environnement de travail (WSL - Windows Subsystem for Linux) :

— **Droits d'exécution sous WSL (`os error 13`) :** Lors de la compilation via

`cargo build`, le système renvoyait fréquemment une erreur de type "Permission denied". Cela était dû à la gestion des permissions NTFS entre Windows et le sous-système Linux, bloquant l'exécution des scripts de *build* de certaines crates comme `serde` ou `syn`.

— **Latence et "Underruns" ALSA :** La lecture simultanée de plusieurs pistes audio entraînait de forts ralentissements et des saccades. La console affichait l'erreur `ALSA lib pcm.c:8568:(snd_pcm_recover) underrun occurred`. Le moteur ALSA sous l'environnement virtualisé WSL n'arrivait pas à fournir les tampons de données (buffers) assez rapidement à la carte son lors de l'ouverture de plusieurs `Sinks` simultanés.

— **Saturation numérique (Clipping) :** Lors de l'addition des signaux de plusieurs pistes pour l'exportation du mix final, l'amplitude globale dépassait fréquemment la limite des valeurs flottantes de l'audio numérique, provoquant une distorsion désagréable et incontrôlée.

— **Gestion des offsets temporels :** Placer les enregistrements avec le bon décalage dans le fichier final nécessitait une conversion mathématique précise entre le temps en secondes, le *Sample Rate* (taux d'échantillonnage), et le nombre de canaux (stéréo/mono).

9.3 Solutions apportées

Pour surmonter ces obstacles techniques, nous avons adopté plusieurs stratégies d'optimisation et de contournement :

— **Résolution des conflits WSL :** Nous avons rapatrié l'ensemble du projet sur la partition native Linux (`ext4`) de WSL plutôt que sur le point de montage `/mnt/c/`, et appliqué des droits stricts (`chmod -R +x`) sur le répertoire de compilation. De plus, nous avons forcé le routage audio vers Windows via **PulseAudio**.

— **Optimisation et Compilation Release :** Pour contrer les erreurs "underrun" d'ALSA, nous avons basculé la compilation en mode optimisé (`cargo run -release`). La suppression des vérifications de débogage a drastiquement accéléré les itérations de notre itérateur `DSPFilter`, permettant à ALSA de recevoir les *samples* audio à temps.

— **Mixage hors-ligne et Hard-Limiting :** Plutôt que de router plusieurs flux vers la carte son lors de l'export, la fonction `export_recorded_session` effectue un mixage "hors-ligne" directement en mémoire. Avant l'écriture via `hound`, nous avons implémenté un système de *Hard-Limiting* (`.max(-1.0).min(1.0)`) pour garantir qu'aucun échantillon ne dépasse le plafond d'amplitude numérique.

— **Alignement binaire des échantillons :** Pour la recomposition temporelle, nous avons forcé l'alignement des échantillons sur la taille des blocs multicanaux (`(raw_offset / channels) * channels`), évitant ainsi le déphasage ou l'inversion des canaux gauche/droite lors du mixage de pistes stéréo.

10 Axes d'amélioration

Bien que le projet réponde de manière robuste aux exigences initiales, nous avons identifié plusieurs axes d'évolution pour transformer **MixRust** en une application encore plus aboutie et ergonomique pour les prochaines échéances :

— **Refonte de l'interface (Navigation par vues) :** Afin d'éviter de surcharger l'écran principal au fur et à mesure que de nouvelles fonctionnalités sont ajoutées, nous prévoyons de repenser l'interface visuelle sur un modèle similaire à des diapositives (façon PowerPoint) ou avec un système d'onglets et de rubans. Cela permettra d'organiser les outils par catégorie (Mixage, Édition, Effets) et de conserver un espace de travail clair et lisible.
— **Édition audio temporelle (Couper/Rogner) :** L'ajout d'outils de manipulation destructive et non-destructive permettra à l'utilisateur de scinder une piste en plusieurs morceaux, de supprimer les silences ou d'extraire une boucle (loop) spécifique d'un échantillon audio.
— **Système de "Waypoints" (Automatisation ciblée) :** Nous souhaitons implémenter des marqueurs temporels interactifs directement plaçables sur la *waveform*. Cela permettra d'appliquer nos filtres DSP (comme la distorsion, la réverbération ou des fondus de volume) de manière chirurgicale, uniquement entre un point A et un point B du fichier, offrant ainsi un véritable système d'automation digne des DAW professionnels.
— **Enregistrement vocal (Capture d'entrée) :** L'intégration de la capture matérielle via le microphone de l'utilisateur permettra d'enregistrer des pistes de voix ou d'instruments directement au sein de l'application, afin de les mixer immédiatement avec les pistes importées.
— **Déploiement et Amélioration du site web :** Pour faciliter la distribution de notre logiciel, nous créerons un véritable fichier d'installation (installeur exécutable) qui sera mis à disposition sur notre site vitrine. En parallèle, le site web lui-même bénéficiera d'une refonte visuelle et de nouvelles sections pour offrir une documentation complète et un guide de prise en main du logiciel.

11 Conclusion

Ce projet **MixRust** nous a permis de faire la transition d'un langage bas niveau classique (C) vers un langage moderne et sécurisé (Rust), tout en nous confrontant aux exigences strictes du développement d'applications multimédias en temps réel.

En partant de zéro, nous avons réussi à bâtir une architecture complexe alliant une interface graphique réactive, des mathématiques appliquées au traitement du signal (DSP), et une logique d'E/S (Entrées/Sorties) robuste. Le choix de concevoir un système d'enregistrement basé sur la capture d'événements plutôt que sur la copie de flux a été une réussite : cela rend notre DAW non-destructif, léger en mémoire, et extrêmement précis lors de l'exportation finale.

Malgré les obstacles techniques liés à l'interopérabilité audio de WSL et à la synchronisation des threads, nous avons su adapter notre code et optimiser notre logique mathématique pour délivrer un logiciel fonctionnel et stable. Au-delà des compétences purement techniques (GTK, Rodio, manipulation de *buffers* audio), ce projet a consolidé notre capacité à planifier un développement de manière itérative, à débugger des environnements virtualisés complexes, et à répartir efficacement les tâches au sein de l'équipe.

Cette expérience nous laisse avec une excellente compréhension de l'ingénierie logicielle audio, prête à être réinvestie dans de futurs défis technologiques.